



Connect Level Order Siblings (medium)

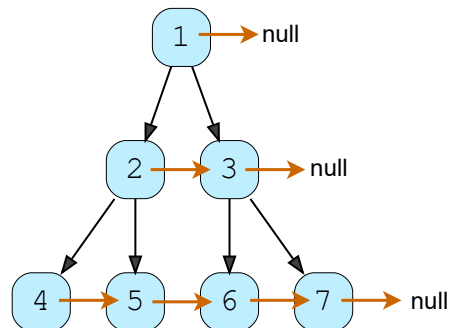
We'll cover the following

- Problem Statement
- Try it yourself
- Solution
- Code
- Time complexity
- Space complexity

Problem Statement

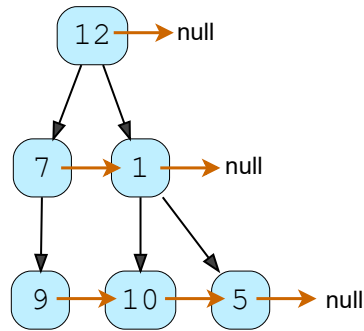
Given a binary tree, connect each node with its level order successor. The last node of each level should point to a **null** node.

Example 1:



Example 2:





Try it yourself

Try solving this question here:



Java



Python3



JS



C++

```

1  import java.util.*;
2
3  class TreeNode {
4      int val;
5      TreeNode left;
6      TreeNode right;
7      TreeNode next;
8
9      TreeNode(int x) {
10         val = x;
11         left = right = next = null;
12     }
13
14     // level order traversal using 'next' pointer
15     void printLevelOrder() {
16         TreeNode nextLevelRoot = this;
17         while (nextLevelRoot != null) {
18             TreeNode current = nextLevelRoot;
19             nextLevelRoot = null;
20             while (current != null) {
21                 System.out.print(current.val + " ");
22                 if (nextLevelRoot == null) {
23                     if (current.left != null)
24                         nextLevelRoot = current.left;
25                     else if (current.right != null)
26                         nextLevelRoot = current.right;
27                 }
28                 current = current.next;
29             }
30             System.out.println();
31         }
32     }
33 };
34
35 class ConnectLevelOrderSiblings {
36     public static void connect(TreeNode root) {
37         // TODO: Write your code here
38     }
39
40     public static void main(String[] args) {
41         TreeNode root = new TreeNode(12);
42         root.left = new TreeNode(7);

```

```

43     root.right = new TreeNode(1);
44     root.left.left = new TreeNode(9);
45     root.right.left = new TreeNode(10);
46     root.right.right = new TreeNode(5);
47     ConnectLevelOrderSiblings.connect(root);
48     System.out.println("Level order traversal using 'next' pointer: ");
49     root.printLevelOrder();
50 }
51 }
52

```

Run

Save

Reset



Solution

This problem follows the [Binary Tree Level Order Traversal](#) pattern. We can follow the same **BFS** approach. The only difference is that while traversing a level we will remember the previous node to connect it with the current node.

Code

Here is what our algorithm will look like; only the highlighted lines have changed:

Java

Python3

C++

JS

```

1  import java.util.*;
2
3  class TreeNode {
4      int val;
5      TreeNode left;
6      TreeNode right;
7      TreeNode next;
8
9      TreeNode(int x) {
10         val = x;
11         left = right = next = null;
12     }
13
14     // level order traversal using 'next' pointer
15     public void printLevelOrder() {
16         TreeNode nextLevelRoot = this;
17         while (nextLevelRoot != null) {
18             TreeNode current = nextLevelRoot;
19             nextLevelRoot = null;
20             while (current != null) {
21                 System.out.print(current.val + " ");
22                 if (nextLevelRoot == null) {
23                     if (current.left != null)
24                         nextLevelRoot = current.left;
25                     else if (current.right != null)
26                         nextLevelRoot = current.right;
27                 }
28                 current = current.next;
29             }
30             System.out.println();
31         }
32     }
33 };
34
35 class ConnectLevelOrderSiblings {
36     public static void connect(TreeNode root) {
37         if (root == null)

```



```

37     if (root == null)
38         return;
39
40     Queue<TreeNode> queue = new LinkedList<>();
41     queue.offer(root);
42     while (!queue.isEmpty()) {
43         TreeNode previousNode = null;
44         int levelSize = queue.size();
45         // connect all nodes of this level
46         for (int i = 0; i < levelSize; i++) {
47             TreeNode currentNode = queue.poll();
48             if (previousNode != null)
49                 previousNode.next = currentNode;
50             previousNode = currentNode;
51
52             // insert the children of current node in the queue
53             if (currentNode.left != null)
54                 queue.offer(currentNode.left);
55             if (currentNode.right != null)
56                 queue.offer(currentNode.right);
57         }
58     }
59 }
60
61 public static void main(String[] args) {
62     TreeNode root = new TreeNode(12);
63     root.left = new TreeNode(7);
64     root.right = new TreeNode(1);
65     root.left.left = new TreeNode(9);
66     root.right.left = new TreeNode(10);
67     root.right.right = new TreeNode(5);
68     ConnectLevelOrderSiblings.connect(root);
69     System.out.println("Level order traversal using 'next' pointer: ");
70     root.printLevelOrder();
71 }
72 }
73

```

Run

Save

Reset



Time complexity

The time complexity of the above algorithm is $O(N)$, where 'N' is the total number of nodes in the tree. This is due to the fact that we traverse each node once.

Space complexity

The space complexity of the above algorithm will be $O(N)$, which is required for the queue. Since we can have a maximum of $N/2$ nodes at any level (this could happen only at the lowest level), therefore we will need $O(N)$ space to store them in the queue.